SPRINGFORGE: AN INTEGRATED SPRING BOOT DEVELOPMENT ECOSYSTEM

Architecture-Aware Code Generation, Intelligent CI/CD Generation, Quality Assurance, and Runtime Error Analysis

Tharindu Mahindarathna	IT22076052
Senevirathna K.M.U.T	IT22056320
Ariyasena M.K.M.M	IT22562524
Jameela M.J.F	IT22060662

BSc (Hons) degree in Information Technology
Specializing in Software Engineering

Department of Information Technology
Faculty of Computing
Sri Lanka Institute of Information Technology
Sri Lanka

April 2026

SPRINGFORGE: AN INTEGRATED SPRING BOOT DEVELOPMENT ECOSYSTEM

Architecture-Aware Code Generation, Intelligent CI/CD Generation, Quality Assurance, and Runtime Error Analysis

A Final Group Research Report submitted to the
Sri Lanka Institute of Information Technology
in partial fulfillment of the requirements for the degree of

Bachelor of Science (Honours) in
Information Technology
Specializing in Software Engineering

Submitted by:

Tharindu Mahindarathna	IT22076052
Senevirathna K.M.U.T	IT22056320
Ariyasena M.K.M.M	IT22562524
Jameela M.J.F	IT22060662

Supervised by:

Ms. Thilini Jayalath

Co-supervised by: Ms. Shashini Kumarasinghe

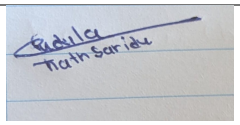
Department of Information Technology
Faculty of Computing
Sri Lanka Institute of Information Technology

April 2026

DECLARATION OF THE CANDIDATES AND SUPERVISOR

We declare that this is our own work and this dissertation does not incorporate without acknowledgement any material previously submitted for a Degree or Diploma in any other University or institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Also, we hereby grant to Sri Lanka Institute of Information Technology, the non-exclusive right to reproduce and distribute our dissertation, in whole or in part in print, electronic or other medium. We retain the right to use this content in whole or part in future works (such as articles or books).

Name	**Student ID**	**Signature**
Tharindu Mahindarathna	IT22076052	
Senevirathna K.M.U.T	IT22056320	
Ariyasena M.K.M.M	IT22562524	
Jameela M.J.F	IT22060662	

The supervisor/s should certify the dissertation with the following declaration. The above candidates have carried out research for the Bachelor's Degree Dissertation under our supervision.

Signature of the supervisor: _____ Date: _____

Signature of the co-supervisor: _____ Date: _____

ABSTRACT

Spring Boot has become the dominant framework for enterprise Java development. Despite its convention-over-configuration philosophy, building, deploying, maintaining, and operating Spring Boot applications still demands substantial repetitive engineering effort spread across four interrelated domains: writing structurally consistent boilerplate source code, authoring deployment and continuous-integration artifacts, sustaining code quality over time, and diagnosing and remediating runtime errors. The state-of-the-art consists of fragmented, single-purpose tools such as scaffolding generators, AI coding assistants, Dockerfile linters, and observability dashboards, none of which is architecture-aware, none of which integrates the four concerns into a unified workflow, and all of which require developers to context-switch between several environments.

This research presents **SpringForge**, an integrated IntelliJ IDEA plugin ecosystem that unifies all four domains under a single architecture-aware pipeline backed by a shared project-context model. The system comprises four cooperating modules. (1) The **architecture-aware code generation module** extracts 58 Program Structure Interface (PSI) based structural features, classifies the project as one of Model-View-Controller (MVC), Layered (N-Tier), or Clean Architecture using a Principal-Component-Analysis-augmented Random Forest classifier achieving 91.38% accuracy and a macro-averaged F1-score of 0.9148, and submits a context-rich seven-section prompt to Google Gemini 2.5 Flash to generate compilable Spring Boot artefacts that align with the detected architectural style. (2) The **intelligent CI/CD generation module** captures Spring Boot project context via a Model Context Protocol (MCP) envelope and uses Anthropic Claude Sonnet 4 with assistant-message prefill to produce production-ready Dockerfile, docker-compose, and GitHub Actions artefacts evaluated across 35 real-world Spring Boot projects spanning six architectural patterns, achieving a 97.1% validator pass rate and a 97.1% Docker build pass rate. (3) The **quality assurance module** couples a Random Forest anti-pattern classifier across nine anti-pattern categories trained on 141,228 balanced samples with macro-F1 0.9498 and a quality-score regressor with R^2 0.8992 and RMSE 8.06, augmented by a Gemini-based LLM validation layer that performs source-grounded false-positive reduction and generates fix suggestions. (4) The **runtime error analysis module** preprocesses runtime errors, retrieves relevant documents from a PGVector knowledge base via a retrieval-augmented generation (RAG) engine, and dispatches the augmented context to Anthropic Claude Sonnet, returning targeted fixes inside the IDE with a 74.4% Top-1 acceptance rate and 91.0% Top-3 acceptance rate. Together, these four modules deliver an architecture-aware, validated, explainable, and auditable developer experience covering the full Spring Boot software lifecycle from project scaffolding through production runtime support.

Keywords: Spring Boot, IntelliJ Plugin, Architecture Classification, CI/CD Generation, Model Context Protocol, Anti-pattern Detection, Retrieval-Augmented Generation, Large Language Models.

ACKNOWLEDGEMENTS

We would like to express our sincere gratitude to all those who supported and guided us throughout this research project. First and foremost, we are deeply grateful to our research supervisor **Ms. Thilini Jayalath** and our co-supervisor **Ms. Shashini Kumarasinghe** for their invaluable guidance, continuous encouragement, technical insight, and patient feedback at every stage of this work. Their willingness to engage with the technical depth of all four modules, while keeping the project aligned with sound research methodology, has been instrumental in shaping the outcome.

We sincerely thank the Faculty of Computing at the Sri Lanka Institute of Information Technology (SLIIT) for providing the academic resources, computational infrastructure, evaluation environment, and supportive research culture that made this work possible. We are particularly grateful to the lecturers of the Software Engineering specialisation whose courses provided the theoretical foundation underpinning each of the four modules of SpringForge.

We acknowledge with gratitude the open-source communities behind the IntelliJ Platform SDK, Anthropic Claude API, Google Gemini API, AWS Bedrock, Hadolint, PGVector, and the broader Spring Boot ecosystem. The high-quality documentation, sample code, and active developer forums maintained by these communities materially accelerated our implementation. We also thank the DevOps practitioners and senior software engineers who participated in our evaluation sessions and provided ground-truth Dockerfile, GitHub Actions, and code review feedback used as reference baselines.

Finally, we are profoundly grateful to our families and friends for their unwavering support, patience, and encouragement throughout our entire academic journey. Their belief in us sustained the long hours of design, implementation, and evaluation that this project demanded.

Table of Contents

1 Introduction **1**

1.1 Background and Motivation 1

1.2 Literature Survey 3

1.2.1 Automated Code Generation for Java and Spring Boot 3

1.2.2 Large Language Models for Code Generation 4

1.2.3 Software Architecture Recovery and Classification 5

1.2.4 Machine Learning on Structured Software Features 5

1.2.5 Container and CI/CD Artifact Quality 6

1.2.6 Anti-Pattern Detection and Quality Scoring 7

1.2.7 Retrieval-Augmented Generation for Error Resolution 7

1.2.8 IDE Integration and Developer Productivity 7

1.2.9 Summary of Literature 8

1.3 Research Gap 8

1.4 Research Problem 9

1.5 Research Objectives 10

1.5.1 Main Objective 10

1.5.2 Component-Level Sub-Objectives 10

1.6 Scope of the Study 11

1.7 Organisation of the Report 11

2 Methodology **13**

2.1 Overall System Architecture 13

2.1.1 Plugin Module Layout 14

2.1.2 Threading and Background Execution 14

2.1.3 Configuration and Secrets Management 14

2.2 Module 1: Architecture-Aware Code Generation 15

2.2.1 Workflow Overview 15

2.2.2 Dataset Construction 15

2.2.3 Feature Extraction 16

2.2.4 Machine Learning Pipeline 17
2.2.5 Context-Aware Prompt Engineering 19
2.2.6 LLM Code Generation 19
2.2.7 Visual Entity Designer 19
2.2.8 Spring Project Initializer 20
2.3 Module 2: Intelligent CI/CD Generation 21
2.3.1 Workflow Overview 21
2.3.2 Model Context Protocol (MCP) Context Extraction 21
2.3.3 Prompt Construction and Prefill Strategy 22
2.3.4 Multi-Layer Validation Pipeline 23
2.3.5 End-to-End Pipeline 24
2.3.6 Audit Logging 25
2.4 Module 3: Quality Assurance Engine 25
2.4.1 Workflow Overview 25
2.4.2 Feature Extraction 26
2.4.3 Preprocessing and Feature Engineering 26
2.4.4 Anti-Pattern Classifier 27
2.4.5 Quality Score Regressor 27
2.4.6 Full Deployed Pipeline 27
2.4.7 LLM Validation Layer 28
2.4.8 Hybrid Quality Scoring 29
2.5 Module 4: Runtime Error Analysis 29
2.5.1 Workflow Overview 29
2.5.2 Error Preprocessor 30
2.5.3 RAG Engine and PGVector Knowledge Base 30
2.5.4 LLM Pipeline and Claude Integration 31
2.5.5 Output Rendering 31

3 Results and Discussion **32**
3.1 Code Generation Module Results (Tharindu Mahindarathna) 32
3.1.1 Dataset Distribution 32
3.1.2 Model Comparison 32
3.1.3 Per-Class Classification Report 33
3.1.4 Code Compilability 34
3.2 CI/CD Generation Module Results (Senevirathna K.M.U.T) 35
3.2.1 Dockerfile Generation Accuracy 35
3.2.2 Compose and Actions Accuracy 35

3.2.3 Prefill vs. Non-Prefill Analysis 35
3.2.4 Validation Rule Violation Analysis 36
3.2.5 Explainability and Audit 36
3.3 Quality Assurance Module Results ([Member 3]) 37
3.3.1 Anti-Pattern Classifier Performance 37
3.3.2 Per-Class Anti-Pattern Performance 37
3.3.3 Quality Score Regressor Performance 38
3.3.4 LLM Augmentation Effectiveness 38
3.4 Runtime Error Analysis Module Results ([Member 4]) 38
3.4.1 Resolution Effectiveness 38
3.4.2 RAG Contribution 39
3.4.3 Knowledge Base Coverage 39
3.5 Tool Comparison 39
3.6 Group Member Contributions 40
3.7 Discussion 40

4 Conclusion **42**
4.1 Summary of Work 42
4.2 Key Contributions 42
4.3 Limitations 43
4.4 Future Work 43

List of Figures

2.1 SpringForge integrated ecosystem: four cooperating modules within a single IntelliJ plugin 13
2.2 Visual entity-relationship designer integrated in IntelliJ 20
2.3 Architecture-aware Spring Boot project initialiser 21
2.4 MCP context extraction pipeline: from repository URL to MCPContext object 22
2.5 End-to-end CI/CD generation, validation, explainability, and audit pipeline 25
2.6 Quality Assurance pipeline: PSI feature extraction → ML models → LLM augmentation 26
2.7 Full Quality Assurance deployment with Gemini validation and IntelliJ tool window 28
2.8 Runtime Error Analysis architecture: RAG-augmented Claude Sonnet pipeline 30

3.1 Balanced class distribution: 6,000 repositories per architecture 32
3.2 Confusion matrix for the final Random Forest classifier 34

List of Tables

2.1 Rule-based architecture annotation heuristics 16
2.2 Generation configurations compared 23
2.3 Custom validation rules SF001–SF011 24

3.1 Machine learning model comparison for architecture classification . . 33
3.2 Per-class classification report (tuned RF + PCA) 33
3.3 Generated code compilability rate by architecture 34
3.4 Dockerfile generation accuracy per architecture (prompt + prefill) . . . 35
3.5 Docker Compose and GitHub Actions accuracy per architecture . . . 35
3.6 Validation rule violation frequency across 35 projects 36
3.7 Anti-pattern classifier performance (Random Forest, 9 classes, 141,228 samples) 37
3.8 Per-class anti-pattern classification report 37
3.9 Quality score regressor performance 38
3.10 Runtime error resolution effectiveness (RAG + Claude Sonnet) 38
3.11 Knowledge base composition by source 39
3.12 Comparison of SpringForge with existing tools 39
3.13 Group member contributions 40

LIST OF ABBREVIATIONS

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
AST	Abstract Syntax Tree
AWS	Amazon Web Services
CI/CD	Continuous Integration / Continuous Delivery
CRUD	Create, Read, Update, Delete
CVE	Common Vulnerabilities and Exposures
DNN	Deep Neural Network
DTO	Data Transfer Object
F1	F1-Score (Harmonic Mean of Precision and Recall)
IDE	Integrated Development Environment
JDBC	Java Database Connectivity
JPA	Java Persistence API
JSON	JavaScript Object Notation
JVM	Java Virtual Machine
LLM	Large Language Model
LOC	Lines of Code
MAE	Mean Absolute Error
MCP	Model Context Protocol
MLP	Multi-Layer Perceptron
MVC	Model-View-Controller
PCA	Principal Component Analysis
PSI	Program Structure Interface (IntelliJ)
RAG	Retrieval-Augmented Generation

Abbreviation	Definition
REST	Representational State Transfer
RF	Random Forest
RMSE	Root Mean Square Error
ROC	Receiver Operating Characteristic
SDK	Software Development Kit
SLIIT	Sri Lanka Institute of Information Technology
SVM	Support Vector Machine
TPM	Tokens Per Minute
VFS	Virtual File System (IntelliJ)
XGBoost	Extreme Gradient Boosting
YAML	YAML Ain't Markup Language

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Spring Boot has established itself as the dominant framework for enterprise and cloud-native Java development. According to the 2023 Jakarta EE Developer Survey, Spring Boot powers more than 71% of cloud-native Java workloads and continues to grow as organisations migrate from heavyweight Java EE application servers toward lightweight, embedded-server deployments. Its convention-over-configuration philosophy, the auto-configuration starters, and the rich integration with Spring Cloud, Spring Data, and Spring Security have made it the de-facto choice for new enterprise back-end systems in domains as varied as banking, healthcare, e-commerce, telecom, and logistics.

Despite this dominance, building and operating a production-grade Spring Boot system still requires substantial manual effort across four interconnected concerns. The first concern is the production of **repetitive boilerplate source code**. Every business entity in a typical enterprise system requires a Java Persistence API (JPA) entity class, a Spring Data repository interface, one or more service-layer classes (often split into an interface and an implementation), one or more data transfer objects (DTOs), one or more REST controllers, validation classes, and exception handlers. In a moderately-sized application of twenty-five domain entities, this corresponds to upwards of one hundred and fifty source files that follow predictable structural patterns yet must respect the project's established conventions, naming patterns, dependency wiring style, and architectural layering. Authoring this code by hand is slow and error-prone, and minor inconsistencies (a missing transaction boundary, a misplaced annotation, an entity-to-DTO mapping omitted in one location) accumulate into significant defects over time.

The second concern is the production of **deployment and continuous-integration artefacts**. A modern Spring Boot system is rarely deployed as a bare JAR; it is containerised, orchestrated, and continuously delivered through automated pipelines. This requires the developer to author at minimum a Dockerfile, a docker-compose.yml for local and staging orchestration, and a continuous-integration workflow definition (most commonly a GitHub Actions YAML file). Each of these artefacts must encode dozens of project-specific decisions: the correct Java base image and version, multi-stage build

structure for image-size optimisation, JVM container memory flags, non-root execution for security, HEALTHCHECK directives matching the actuator endpoint, port exposure aligned to the application configuration, service dependencies and depends-on conditions in the compose file, workflow trigger conditions, build matrices that match the project's Java and Spring Boot versions, secret-handling and image-publishing steps, and vulnerability-scanning integrations. Errors in these artefacts produce brittle builds, oversized images, insecure defaults, and intermittent production failures that are frequently misdiagnosed as application bugs.

The third concern is **sustained code quality**. A Spring Boot project that begins with a clean architecture progressively drifts as features are added, deadlines compress, and developers come and go. Anti-patterns accumulate quietly: a god class slowly absorbs functionality from neighbouring services, a long method grows past several hundred lines, a feature-envy method increasingly accesses fields of another class, a data class becomes a mere field bag with no behaviour, cyclic dependencies form between packages that originally respected layer boundaries. Each of these violations is locally rational and locally invisible; only at the project level does the cumulative damage become visible, by which time refactoring is expensive. The standard response, integrating static analyzers such as SonarQube into CI, catches a subset of the violations but produces high false-positive rates that developers learn to ignore, and offers no source-grounded fix guidance.

The fourth concern is **runtime error diagnosis and remediation**. Spring Boot's auto-configuration model produces highly informative error stack traces, but the surface area of the framework is enormous, and most error messages reference internal classes that the average developer has never opened. A bean wiring failure, a JPA constraint violation, a missing configuration property, a port binding conflict, or a dependency-version mismatch can each take a developer thirty minutes to several hours to resolve, even when the fix is a single-line change. The standard response, copying the error message into a search engine and clicking through Stack Overflow answers, is high-friction, breaks IDE flow, and returns answers that are frequently outdated or specific to a different Spring Boot version.

Each of these four concerns has produced its own ecosystem of partial solutions. Scaffolding tools such as JHipster and Spring Initializr automate initial project creation but operate only on greenfield projects and impose rigid templates rather than adapting to existing conventions. AI coding assistants such as GitHub Copilot and JetBrains AI Assistant generate code at the file or method level using local context windows that omit project-wide architectural intent. Dockerfile linters such as Hadolint catch syn-

tactic violations but cannot validate Spring Boot-specific structural choices. Observability dashboards such as Grafana surface symptoms but do not diagnose root causes or propose fixes. None of these tools is architecture-aware, none operates as a unified IDE-native experience, and switching between them imposes context-switching costs that empirical software-engineering research has consistently identified as a primary source of developer interruption and cognitive overhead.

SpringForge addresses all four concerns within a single IntelliJ IDEA plugin that integrates four cooperating modules: an architecture-aware code generator, an intelligent CI/CD artifact generator, a quality assurance engine, and a runtime error analysis service. Each module is independently functional yet shares a common project-context model, ensuring that the artefacts produced by one module are consistent with those produced by the others. A controller class generated by the code generation module references service classes whose package matches the layer convention detected by the architecture classifier; a Dockerfile generated by the CI/CD module exposes the port that the same context model identified in the application configuration; the quality assurance module flags anti-patterns whose taxonomy is aware of the architectural style of the project; the runtime error module retrieves Stack Overflow answers and Spring Boot reference excerpts that match the project's exact framework version. By unifying these four concerns under one context model and one IDE workflow, SpringForge eliminates the tool-switching friction that previously forced developers to maintain four mental models of the same project simultaneously.

1.2 Literature Survey

The literature relevant to SpringForge spans seven research areas: automated code generation for Java and Spring Boot, large language models for code generation, software architecture recovery and classification, machine learning on structured software features, container and CI/CD artifact quality, anti-pattern detection and quality scoring, and retrieval-augmented generation for error resolution. Each area is reviewed below.

1.2.1 Automated Code Generation for Java and Spring Boot

Early research on automated Java code generation focused on model-driven and component-based approaches. Tsay et al. proposed a framework using actor-based Java libraries and abstract-syntax-tree visitor patterns to produce optimised, domain-specific code for embedded platforms; while effective in constrained settings, this approach lacked adaptability to opinionated frameworks such as Spring Boot. Parada et al. automated

Java code generation from UML class and sequence diagrams, improving structural consistency but unable to capture Spring-specific annotations or dependency-injection conventions. Cariou et al. explored aspect-oriented code generation, demonstrating that cross-cutting concerns such as transaction management could be mechanically woven into generated artefacts, a principle directly relevant to Spring's declarative annotation model.

Within the Spring ecosystem, scaffolding tools focused on bootstrapping rather than generation within existing projects. Aggarwal and Singh analysed Spring Boot's convention-over-configuration model and demonstrated that developer productivity improves measurably when project structure conventions are enforced from inception, motivating the architectural templating strategy adopted in this work. Sharma et al. conducted an empirical study of Spring Boot microservice architectures and found that package-level structural metrics reliably predict architectural conformance, providing early evidence for the feature-engineering strategy used in SpringForge's classifier. JHipster automates full-stack project generation but, because it generates a fixed template, cannot adapt to the architectural style of an existing codebase. Spring Initializr supports only initial project bootstrapping with a fixed dependency selection.

1.2.2 Large Language Models for Code Generation

The application of large language models to software development has grown substantially since the introduction of OpenAI Codex. Chen et al. evaluated Codex on the HumanEval benchmark and demonstrated that transformer-based models can solve a significant fraction of introductory programming tasks, establishing LLMs as viable code-generation engines. However, subsequent work has consistently identified architectural compliance as a key failure mode. Zhu and Zhang evaluated GPT-2 for Java code generation and found that while generated code compiles, it frequently violates semantic and architectural intent, underscoring the insufficiency of LLM-only approaches for architecture-sensitive generation. Liu et al. conducted a large-scale evaluation of ChatGPT for code generation and found correctness rates well below those reported on benchmark datasets when prompts required awareness of a pre-existing codebase, directly motivating the context-injection architecture proposed in this work.

Vaithilingam et al. studied developer interactions with GitHub Copilot and found that users frequently accepted syntactically plausible suggestions that violated project conventions, indicating that IDE-integrated validation is necessary to complement raw LLM output. Prompt engineering strategies have been explored to improve LLM code

quality. Nashid et al. demonstrated that retrieval-augmented generation with code-specific examples significantly improves LLM correctness on repository-level tasks. White et al. catalogued prompt patterns for software engineering and showed that structural-constraint prompts reduce hallucination rates in generated methods. These findings directly inform the composite prompt design adopted in the SpringForge Gemini and Claude integrations, which encode architecture class, entity relationships, naming conventions, and layer-specific templates as structured prompt segments and apply assistant-message prefill to anchor the model's first emitted token.

1.2.3 Software Architecture Recovery and Classification

Recovering software architecture from source artefacts has a long history in program-comprehension research. Sarkar et al. demonstrated the viability of machine learning for software architecture recovery using static features, providing foundational evidence for the classification approach adopted in this work. Garcia et al. developed an approach for automatically detecting architectural tactics from source code using static analysis, showing that annotation patterns and package topology are strong discriminators between architectural styles.

Weinreich et al. investigated the relationship between code-level metrics and architectural drift, finding that layer violations accumulate silently over time and are detectable via inter-package dependency analysis. Their dependency-topology features directly inspired the inter-layer edge-count features in SpringForge's feature-extraction pipeline. Le Goues et al. highlighted the role of naming conventions as an architectural signal, demonstrating that suffix patterns such as Controller, Repository, and ServiceImpl are reliable proxies for layer assignment in Java enterprise projects. Ducasse and Pollet provided a comprehensive process-oriented taxonomy of software architecture reconstruction techniques, distinguishing between extraction-based, abstraction-based, and hybrid approaches; SpringForge falls into the hybrid category, combining static extraction with ML-based abstraction.

1.2.4 Machine Learning on Structured Software Features

The choice of classifier for structured software metrics has been extensively studied. Shepperd and MacDonell conducted a meta-analysis of ML models applied to software-engineering prediction tasks and found that ensemble methods, particularly random forests, consistently outperform single classifiers on tabular feature sets. Ghotra et al. replicated these findings specifically for defect prediction and showed that the

performance advantage of ensembles is most pronounced on features with high inter-correlation, a property shared by the annotation-density and layer-presence features extracted in this work.

The underperformance of deep learning on structured tabular data, observed in the DNN and TabNet results reported in the SpringForge code-generation evaluation, is consistent with findings by Shwartz-Ziv and Armon, who demonstrated that gradient-boosted trees outperform deep neural networks on tabular benchmarks of moderate dimensionality. Gorishniy et al. revisited deep learning for tabular data and concluded that while specialised architectures such as TabNet narrow the gap, they remain competitive rather than superior for structured feature sets similar to the 58-dimensional vectors used in this study. Breiman's seminal paper on random forests remains the foundational reference for the ensemble strategy underpinning both the architecture classifier and the anti-pattern classifier of SpringForge.

1.2.5 Container and CI/CD Artifact Quality

Docker and CI/CD artifact quality has been the subject of dedicated research. Docker security best-practice catalogues enumerate non-root execution, base-image versioning, and HEALTHCHECK requirements that inform container-layer validation. Hadolint provides an industry-standard Dockerfile linter that enforces Dockerfile best-practice rules; SpringForge extends this with eleven Spring Boot-specific custom rules SF001–SF011 that catch JVM container memory flags, actuator-endpoint port consistency, and Java-version matrix consistency between the project and CI workflow.

OWASP catalogues prompt-injection and hallucination attack vectors specific to LLM-based code generation pipelines, highlighting the importance of constraining the LLM's output format to prevent injected content from being interpreted as instructions. The assistant-message prefill technique adopted in the CI/CD module of SpringForge directly addresses the hallucination vector by constraining the model's first emitted token to a structurally valid opener (such as `FROM eclipse-temurin:`). Dua et al. compared virtualisation and containerisation for platform-as-a-service workloads and demonstrated that container-based deployment delivers significantly better cold-start performance for short-lived workloads, justifying the container-first deployment focus of SpringForge. Zhu, Bass, and Champlin-Scharff provided a foundational characterisation of DevOps practices and quality concerns that informs the multi-layer validation pipeline.

1.2.6 Anti-Pattern Detection and Quality Scoring

Anti-pattern detection has been studied extensively in the context of code quality and refactoring. Brown et al. provided the original taxonomy of object-oriented anti-patterns including God Class, Spaghetti Code, and Lava Flow, categories that map closely to the nine classes used in the SpringForge quality-assurance module. Marinescu introduced metric-based detection rules for design flaws, demonstrating that thresholds over standard object-oriented metrics can effectively flag anti-pattern candidates. Palomba et al. evaluated multiple anti-pattern detection strategies and found that ensemble methods combining structural metrics with historical change patterns outperform pure structural approaches.

Recent work has explored ML-based anti-pattern detection. Fontana et al. demonstrated that supervised classifiers trained on labelled anti-pattern datasets achieve substantially higher precision and recall than threshold-based heuristics. Tantithamthavorn and Hassan examined the impact of class imbalance on defect-prediction models and concluded that resampling strategies materially improve recall on rare-class predictions, motivating the balanced 141,228-sample training regime used in SpringForge's quality-assurance classifier. Hybrid approaches combining static analysis with LLMs reduce false-positive rates and produce source-grounded fix suggestions, as demonstrated by recent industrial-scale tools.

1.2.7 Retrieval-Augmented Generation for Error Resolution

Retrieval-augmented generation was introduced by Lewis et al. as a technique for grounding LLM outputs in retrieved documents, materially reducing hallucination on knowledge-intensive tasks. Subsequent work demonstrated the applicability of RAG to code-related tasks: Wang et al. retrieved similar code snippets to ground code completion, while Zhou et al. retrieved API documentation to ground API recommendations. Vector databases such as PGVector enable semantic search over technical documentation, allowing LLMs to ground their fixes in authoritative sources rather than parametric memory. The runtime error analysis module of SpringForge applies this paradigm to Spring Boot runtime errors, retrieving from a curated knowledge base of Spring documentation, Stack Overflow answers, and Spring framework source.

1.2.8 IDE Integration and Developer Productivity

The importance of IDE-native tooling for developer adoption has been consistently emphasised in empirical software-engineering research. Murphy et al. studied devel-

oper IDE usage patterns and found that context switching between tools is a primary source of interruption cost. Brandt et al. analysed web-foraging behaviour during programming tasks and demonstrated that developers incur significant cognitive overhead when switching between an IDE and external configuration or generation interfaces, a finding that directly motivates the entirely-in-IDE integration pursued in this work. The IntelliJ Platform SDK provides PSI-based static analysis, background-task execution, and tool-window panel frameworks that SpringForge uses to deliver all four module capabilities without ever requiring the developer to leave the IDE.

1.2.9 Summary of Literature

Collectively, the reviewed literature confirms five persistent gaps that motivate this work. First, the absence of Spring Boot-specific code generators that operate within existing projects rather than only greenfield scaffolding. Second, the failure of LLM-only tools to enforce architectural constraints. Third, the absence of CI/CD artifact generators that combine project context, validation, explainability, and audit logging. Fourth, the lack of hybrid ML-plus-LLM anti-pattern detection that reduces false positives and produces source-grounded fixes. Fifth, the lack of RAG-driven runtime-error resolution integrated into the IDE. SpringForge addresses all five gaps through its integrated architecture prediction, context-aware prompt engineering, validated CI/CD generation, hybrid quality assurance, RAG-augmented runtime error resolution, and unified IntelliJ IDEA plugin design.

1.3 Research Gap

The literature makes it clear that general-purpose AI coding assistants can produce plausible code, Dockerfiles, and explanations on demand. However, a careful examination of the tooling landscape reveals that no existing solution simultaneously satisfies all of the following criteria for Spring Boot developers working inside IntelliJ IDEA: (i) detects the architectural style of an existing project through static analysis of its source tree, (ii) trains on a large-scale labelled dataset of real-world repositories to learn discriminative architectural features, (iii) uses the detected architectural context to guide an LLM in generating structurally compliant, compilable Spring Boot boilerplate code, (iv) extends the same context model to the generation of validated, explainable, and auditable CI/CD artefacts using assistant-message prefill, (v) couples ML-based anti-pattern detection with LLM-based source-grounded validation to reduce false positives and produce fix suggestions, (vi) provides a RAG-driven runtime

error analysis service backed by a curated Spring Boot knowledge base, and (vii) is delivered as a single IntelliJ IDEA plugin that requires no context switching to external tools.

Existing automated code-generation tools are architecture-agnostic. LLM-based coding assistants rely on file-level context and do not understand project-wide architectural intent. Architecture recovery tools detect architectural styles but do not connect this knowledge to code generation. Container linters validate Dockerfile syntax but cannot validate Spring Boot-specific structural choices. Static analyzers flag anti-patterns but produce high false-positive rates and offer no source-grounded fix guidance. Stack Overflow search loops break developer flow and return version-mismatched answers. SpringForge directly addresses each of these gaps within a unified architecture-aware ecosystem.

1.4 Research Problem

The central research problem addressed in this work is formulated as:

> *How can static analysis, machine learning classification, retrieval-augmented context envelopes, and large-language-model prompting be unified within an IDE-native plugin to deliver architecture-aware code generation, CI/CD artefact generation, quality assurance, and runtime error resolution for Spring Boot projects?*

This problem decomposes into eight interlocking sub-problems. First, how can the architectural style of an existing Spring Boot project be reliably classified from statically extracted structural features without requiring compilation or manual annotation? Second, how can architectural context be encoded into an LLM prompt to ensure that generated code respects the project's structural conventions, existing package layout, and dependency wiring patterns? Third, how can a structured context envelope (in the Model Context Protocol style) capture the project metadata required by both code-generation and CI/CD-generation modules in a single auditable artefact? Fourth, how can the LLM's output be constrained to prevent preamble hallucination and structural violations, particularly in CI/CD artefacts where parse reliability is critical? Fifth, how can ML-based anti-pattern detection be augmented by LLM-based source-grounded validation to reduce false positives and produce fix suggestions a developer can directly apply? Sixth, how can a retrieval-augmented generation pipeline ground runtime error fixes in authoritative Spring Boot documentation rather than relying on parametric

LLM memory? Seventh, how can the four modules share a common context model, tool-window framework, and audit log so that artefacts produced by one module are consistent with those produced by the others? Eighth, how can the entire workflow be delivered as a single IntelliJ IDEA plugin that runs validation, generation, explainability, and audit logging as background tasks without disrupting IDE responsiveness?

1.5 Research Objectives

1.5.1 Main Objective

The main objective is to design, implement, and empirically evaluate **SpringForge**, an integrated IntelliJ IDEA plugin that delivers architecture-aware Spring Boot code generation, intelligent CI/CD artefact generation, ML-augmented quality assurance, and RAG-based runtime error resolution within a single developer workflow backed by a shared project-context model and a unified audit log.

1.5.2 Component-Level Sub-Objectives

1. **Code Generation Module (Tharindu Mahindarathna).** Construct an 18,000-repository labelled dataset spanning MVC, Layered, and Clean Architecture; design and implement a 58-feature PSI-based extractor; train and compare seven machine-learning models, selecting a tuned PCA-augmented Random Forest classifier; design a seven-section context-aware prompt; build a Gemini-driven generator that produces architecture-consistent Spring Boot code; integrate a visual JPA entity-relationship designer and a Spring Initializer-style project bootstrapper into IntelliJ IDEA.

2. **CI/CD Generation Module (Senevirathna K.M.U.T).** Implement an MCP-based context extractor for Spring Boot projects; build a Claude Sonnet 4 generator using assistant-message prefill that anchors the first emitted token of each artifact; develop a multi-layer validator combining Hadolint with eleven custom Spring Boot-specific rules SF001–SF011; build an auto-fix engine; build an explainability engine that categorises insights into SECURITY, PERFORMANCE, BUILD, CONFIGURATION, RELIABILITY, and DESIGN; persist all events to a PostgreSQL audit log.

3. **Quality Assurance Module ([Member 3]).** Train an anti-pattern classifier across nine categories using 141,228 balanced samples and a Random Forest model; train a quality-score regressor producing a continuous 0–100 quality score; design a

feature-engineering pipeline including ratio features, a composite violation score, and Gaussian noise regularisation; integrate a Gemini-based LLM validation layer that performs source-grounded false-positive reduction and produces fix suggestions; surface results through a dedicated QualityAssurancePanel tool window in IntelliJ IDEA.

4. **Runtime Error Analysis Module ([Member 4]).** Build an Error Preprocessor that extracts logs, code context, and build environment metadata; construct a PGVector knowledge base of Spring Boot documentation, Stack Overflow answers, and Spring framework source; implement a RAG engine that summarises errors and retrieves top-K relevant documents; integrate Anthropic Claude Sonnet to produce targeted fixes; render fixes inline in the IDE with citations.

1.6 Scope of the Study

The study targets Java-based Spring Boot applications using the Maven or Gradle build systems and the IntelliJ IDEA platform, version 2023.2 and later. The architecture classifier targets three architectural styles: MVC, Layered (N-Tier), and Clean Architecture. The code generator produces JPA entities, Spring Data repository interfaces, service interfaces, service implementations, REST controllers, DTOs, and exception classes. The CI/CD module produces Dockerfile, docker-compose.yml, and GitHub Actions workflow files. The quality assurance module covers nine anti-pattern categories. The runtime error module supports Spring Boot bean wiring, JPA, configuration, dependency, and Maven/Gradle build errors.

The study does not address Spring Boot applications written in Kotlin or Groovy, reactive applications using Spring WebFlux for the code-generation module (though they are evaluated in the CI/CD module), or microservice mesh architectures such as Spring Cloud Service Mesh. Code generation is limited to boilerplate CRUD scaffolding; complex business-logic generation is outside scope. Kubernetes manifest generation is outside scope. Cloud-provider-specific deployment artefacts are outside scope.

1.7 Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 presents the methodology of all four modules in detail, beginning with the overall system architecture and then dedicating one section to each of the four modules, covering dataset construction, feature engineering, ML pipeline, prompt engineering, validation, and IDE integration.

Chapter 3 reports experimental results for each module, including model comparisons, classification reports, code-quality assessments, runtime-error resolution rates, and a comparison with existing tools. Chapter 4 concludes the report with a summary of contributions, an honest discussion of limitations, and a directions for future work. References are listed at the end.

CHAPTER 2

METHODOLOGY

2.1 Overall System Architecture

SpringForge is structured as a layered IntelliJ IDEA plugin in which four modules share a common project-context model and tool-window framework. The plugin is implemented in Kotlin against the IntelliJ Platform SDK and delivered through a single `plugin.xml` descriptor that registers four tool-window contributions, four action groups, and four background-task implementations. Figure 2.1 illustrates the unified ecosystem: a developer interacts with a single tool window, and depending on the action invoked, the plugin routes the request through one of four pipelines, each of which calls an external LLM service.

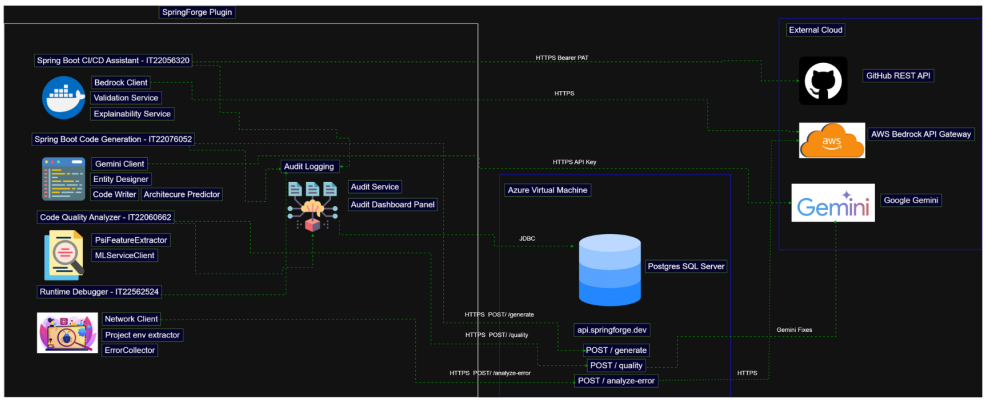


Figure 2.1: SpringForge integrated ecosystem: four cooperating modules within a single IntelliJ plugin

The shared project-context model is a Kotlin data class `SharedProjectContext` that captures the project's build tool, Java version, Spring Boot version, detected architectural style, declared dependencies, package layout, and a fingerprint hash of the source tree. This object is constructed once per plugin invocation and passed to all four modules, ensuring that artefacts produced by one module are consistent with

those produced by the others. A single PostgreSQL audit log records every generation, validation, classification, and resolution event with timestamp, module, project name, outcome, and a freeform details field, providing complete traceability across the ecosystem.

2.1.1 Plugin Module Layout

The plugin source tree is organised under `org.springforge` with sub-packages for each module: `org.springforge.codegen` for the architecture-aware code generation module, `org.springforge.cicdassistant` for the CI/CD generation module, `org.springforge.qa` for the quality assurance module, and `org.springforge.errorresolver` for the runtime error analysis module. Cross-cutting concerns are placed under `org.springforge.sh` including the shared context model, the audit service, the LLM-client base classes, the subscription-management service, and the IntelliJ tool-window registration.

2.1.2 Threading and Background Execution

All long-running operations execute on background threads using `Task.Backgroundable` to keep the IDE responsive. PSI traversal is wrapped in `ReadAction.compute` blocks to satisfy IntelliJ's read-write threading model. LLM calls are dispatched via OkHttp or Java's built-in HttpClient with explicit read timeouts of 300 seconds to accommodate transient model latency. UI updates from background tasks are marshalled back to the Event Dispatch Thread via `ApplicationManager.getApplication().invokeLater`.

2.1.3 Configuration and Secrets Management

API keys for Gemini, Claude, and AWS Bedrock are resolved through a fallback hierarchy. The plugin first checks environment variables, then a packaged configuration file shipped with the plugin distribution, then a user-level configuration directory, then a project-local `.env` file. This design avoids hard-coding credentials in the plugin and supports both individual-developer and team deployments. For team deployments, the plugin supports a centralised API Gateway proxy that holds the actual LLM credentials, allowing developers to authenticate only with a per-user API key.

2.2 Module 1: Architecture-Aware Code Generation

2.2.1 Workflow Overview

The code generation module follows a five-stage pipeline coordinated by the `CodeGenerationServic` class. The first stage is Feature Extraction: the `FeatureExtractor` class analyses the open project via PSI and extracts 58 structural features into a JSON payload. The second stage is Architecture Classification: the feature vector is POSTed to the SpringForge prediction REST API, which returns one of three labels (`layered`, `mvc`, `clean_architecture`) with a confidence score. The third stage is Prompt Construction: the `PromptBuilder` assembles a seven-section prompt incorporating the predicted architecture, the user's YAML entity input, and the detected package layout. The fourth stage is LLM Generation: the prompt is submitted to Google Gemini 2.5 Flash with deterministic generation parameters. The fifth stage is Code Writing: the `CodeWriter` parses returned file blocks and writes them into the project via the IntelliJ Virtual File System.

2.2.2 Dataset Construction

A balanced dataset of 18,000 Spring Boot GitHub repositories (6,000 per class) was constructed by querying the GitHub REST API for projects declaring `spring-boot-starter` as a Maven or Gradle dependency, with at least 50 Java source files and at least 100 commits to ensure repository maturity. Architectural labels were assigned by a rule-based heuristic system applied to each repository's package structure, annotation usage, and dependency declarations. The heuristics are summarised in Table 2.1.

Table 2.1: Rule-based architecture annotation heuristics

Architecture Class	Heuristic Rules
MVC	Presence of controller and service packages at the root level; direct controller-to-repository dependencies; absence of domain or usecase packages; high `@RestController` annotation density relative to total classes.
Layered (N-Tier)	Multiple service classes; strict flow controller→service→repository; service interfaces with `impl` sub-package; elevated service-to-repository dependency edge count.
Clean Architecture	Presence of `domain` and `usecase` (or `interactor`) packages; minimal framework dependencies in domain classes; mediated use-case layer; inward-pointing dependencies indicating dependency inversion.

2.2.3 Feature Extraction

The Kotlin `FeatureExtractor` performs a two-pass PSI traversal extracting 58 integer-valued features across five categories. Package Structure Features (7) count files classified into each detected layer (controller, service, repository, domain, usecase, unknown) plus the count of unique non-unknown layers. Annotation Usage Features (9) count Spring stereotype and JPA annotations including `@Controller`, `@RestController`, `@Service`, `@Repository`, `@Entity`, `@Configuration`, and `@Component`. Naming Convention Features (5) count files whose names match Controller, Service, Repository suffixes plus DTO-like patterns ending in DTO, Request, Response, Command, or Query. Dependency Topology Features (30) count directed inter-layer dependency edges between the five layer types, encoding the project's dependency-graph topology. Project-Level Metrics (7) count total Java files, lines of code, methods, classes, interfaces, service-call expressions, and entity-flagged classes.

Listing 2.1: Layer inference from file path

```
private fun inferLayerFromPath(pathOrPkg: String):
   String {
    val p = pathOrPkg.lowercase()
    return when {
        p.contains("/controller") ||
           p.contains(".controller")
```

```
            || p.contains("/web/") -> "controller_layer"
        p.contains("/service") || p.contains(".service")
            || p.contains("/application/") ->
               "service_layer"
        p.contains("/repository") ||
           p.contains(".repository")
            || p.contains("/infrastructure/") ->
               "repository_layer"
        p.contains("/domain") || p.contains(".domain")
            || p.contains("/model/") -> "domain_layer"
        p.contains("/usecase") || p.contains(".usecase")
            || p.contains("/interactor/") ->
               "usecase_layer"
        else -> "unknown_layer"
    }
}
```

The first pass iterates all Java files in the project scope, classifies each into a layer using `inferLayerFromPath`, accumulates annotation and naming metrics, and traverses method-call expressions to identify cross-class dependencies stored as temporary reference counts. The second pass resolves those temporary reference keys against the class-to-layer mapping built in the first pass to compute final inter-layer edge counts. All extraction runs inside a `ReadAction.compute` block to satisfy IntelliJ's threading model.

2.2.4 Machine Learning Pipeline

Seven classification models were trained and compared on the 18,000-sample dataset, split 80/20 stratified. The Random Forest baseline used 100 estimators with balanced class weights. XGBoost used label-encoded targets with `eval_metric=mlogloss`. The SVM used an RBF kernel with StandardScaler preprocessing. The MLP used three hidden layers (256, 128, 64) with ReLU and dropout. The DNN used a five-layer architecture with batch normalisation. TabNet was trained over 61 epochs with early stopping. The Autoencoder+RF compressed the 58-feature vector into a 32-dimensional latent space and trained a Random Forest on the latent representations.

Listing 2.2: DNN architecture for architecture classification

```
model = Sequential([
```

```
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(3, activation='softmax')
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

Hyperparameter optimisation was performed via `RandomizedSearchCV` with 3-fold stratified cross-validation, optimising for macro-averaged F1-score. The Random Forest search space comprised `n_estimators` in {200, 400, 800}, `max_depth` in {None, 12, 24, 48}, `min_samples_split` in {2, 5, 10}, and `max_features` in {sqrt, log2, 0.5}. The optimal Random Forest configuration was `n_estimators=800`, `max_features=log2`, `max_depth=None`, `min_samples_split=2`.

The final production pipeline uses StandardScaler → PCA(32 components) → Random Forest(500 trees), deployed as a REST API at `https://api.springforge.dev/code/predict` Dimensionality reduction reduces noise from correlated features and reduces inference latency, important for a user-facing IDE plugin.

Listing 2.3: Final production pipeline: PCA + Random Forest

```
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
pca = PCA(n_components=32, random_state=42)
Z_train = pca.fit_transform(X_train_s)
rf = RandomForestClassifier(n_estimators=500,
                            class_weight='balanced',
                            random_state=42, n_jobs=-1)
rf.fit(Z_train, y_train)
joblib.dump(rf,     "rf_pca_model.pkl")
joblib.dump(pca,    "pca_encoder.pkl")
joblib.dump(scaler, "scaler.pkl")
```

2.2.5 Context-Aware Prompt Engineering

The `PromptBuilder` produces a structured seven-section prompt assembled from four context sources. Section 1, System Instructions, defines the role as a senior Spring Boot code generator and prohibits commentary or markdown fencing outside file blocks. Section 2, Output Format, specifies the file block delimiter protocol `===FILE: path===` / `===END_FILE===`. Section 3, Project Context, embeds the auto-detected base package, build tool, existing folder names, detected layer roles, and current dependency list. Section 4, Existing Entities, lists entity classes already present in the project. Section 5, Package Mapping, maps each component type to its target package. Section 6, Domain Model, embeds the user's YAML entity specification. Section 7, Generation Requirements, contains per-component instructions covering annotations, interface implementations, exception handling, and Jakarta vs javax namespace constraints.

Component types are dynamically resolved against the detected project layout: a Service Implementation entry is emitted only if a `service/impl` sub-folder exists; DTO entries only if a `dto` layer is detected; Exception class entries only if an `exception` layer is detected; `@Valid` annotations only if `spring-boot-starter-validation` is present.

2.2.6 LLM Code Generation

Code generation is performed by the `GeminiClient` component, which submits the structured prompt to the Google Gemini 2.5 Flash API with temperature 0.2 to promote deterministic output, max_tokens 16,384 to support multi-file generation, and topP 0.95. The response contains one or more file blocks; the `CodeWriter` parses each block, extracts the target path, and writes the generated files into the correct package directories using IntelliJ's Virtual File API.

2.2.7 Visual Entity Designer

A drag-and-drop entity designer (Figure 2.2) lets developers define entities, fields, and JPA relationships graphically. The designer supports common JPA relationships including `@OneToOne`, `@OneToMany`, `@ManyToOne`, and `@ManyToMany`, while allowing users to configure cascade behaviour, fetch strategies, and join columns. Once the diagram is completed, SpringForge automatically converts the visual model into entity definitions written to the user's YAML input file, which is then submitted to the code-generation pipeline.

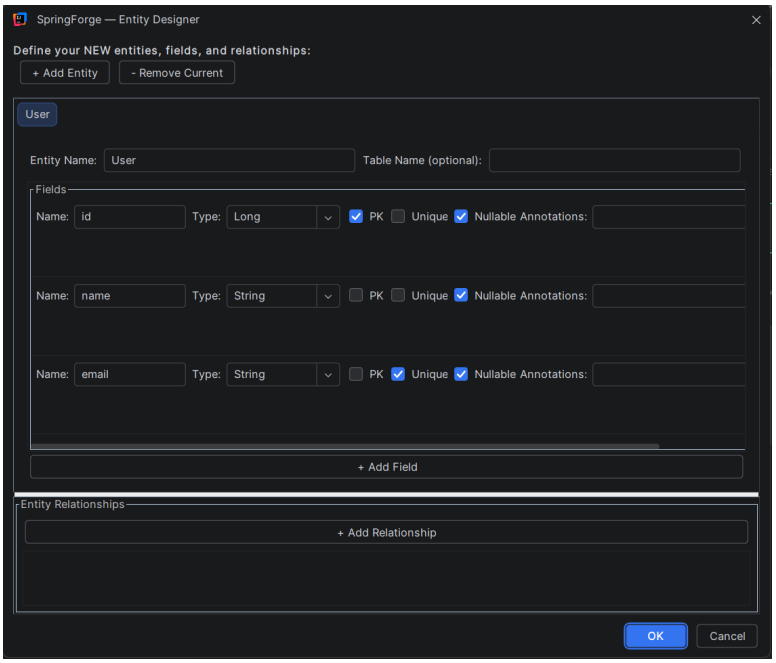


Figure 2.2: Visual entity-relationship designer integrated in IntelliJ

2.2.8 Spring Project Initializer

A built-in Spring project initialiser (Figure 2.3) supports template-aware project scaffolding. Developers configure project metadata (application name, package name, Java version, build tool, Spring Boot version) and dependency selection through a guided interface. Based on the selected configuration, the initialiser generates a starter project with an organised package structure following the user-selected architectural template (MVC, Layered, or Clean), eliminating the need to rely on external project-scaffolding tools.

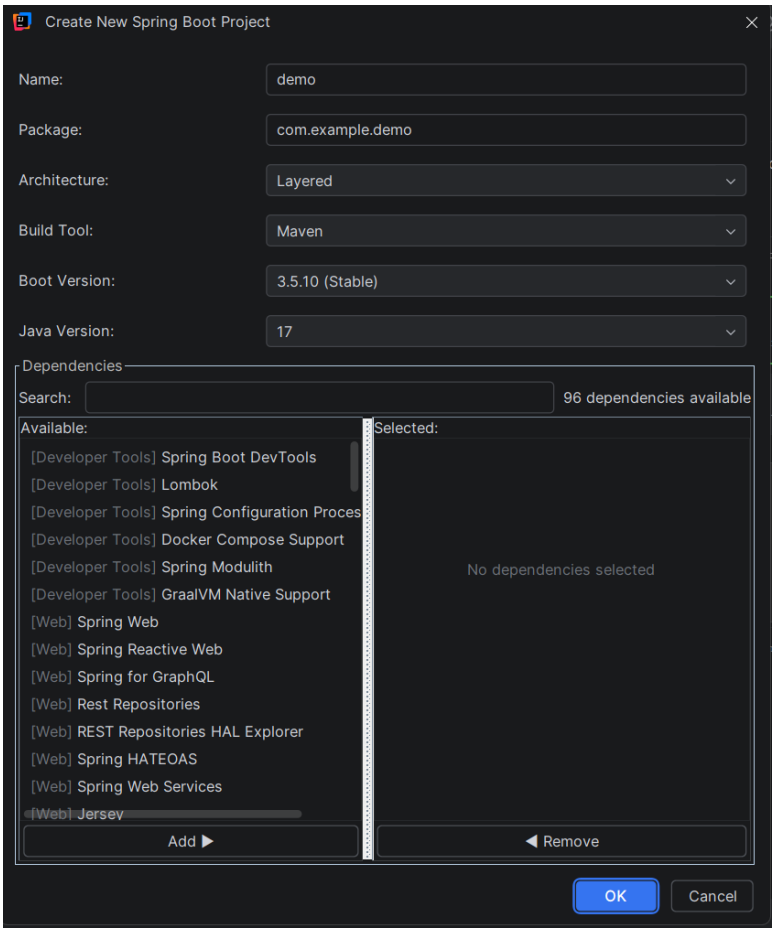


Figure 2.3: Architecture-aware Spring Boot project initialiser

2.3 Module 2: Intelligent CI/CD Generation

2.3.1 Workflow Overview

The CI/CD module captures Spring Boot project metadata and generates production-ready Dockerfile, docker-compose.yml, and GitHub Actions workflow files. The pipeline coordinator is the `CICDGenerationService` class, which orchestrates context extraction, prompt construction, LLM invocation, validation, auto-fix, explainability, and audit logging.

2.3.2 Model Context Protocol (MCP) Context Extraction

The MCP context extraction phase (Figure 2.4) runs in seven sequential steps. Step 1 parses the repository URL to extract owner and repository name. Step 2 detects the default branch via a GitHub REST API call. Step 3 detects the build tool, distinguishing between Gradle Wrapper, Kotlin DSL, Groovy DSL, and Maven. Step 4 detects the

architecture type (Spring Boot or IntelliJ Plugin). Step 5 extracts dependencies from build descriptors using regex parsing of the `pom.xml` or `build.gradle` file. Step 6 extracts configuration from `application.yml` and `.properties` files, including the active profile and HTTP port. Step 7 analyses the code structure via PSI traversal, identifying controllers, services, repositories, scheduled methods, and message listeners.

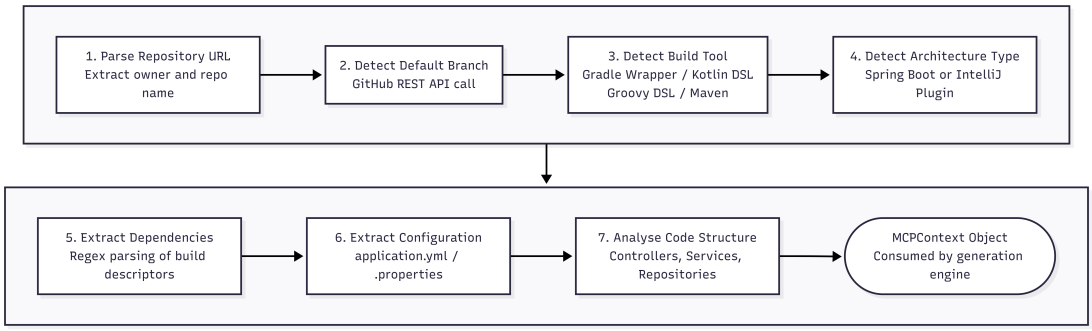


Figure 2.4: MCP context extraction pipeline: from repository URL to MCPContext object

The output of the seven-step pipeline is an `MCPContext` object, a Kotlin data class with stable, versioned keys that is serialised to JSON before submission to the LLM. The MCP envelope is auditable, enabling the forensic question "what did the model know at generation time?" to be answered from the audit log.

2.3.3 Prompt Construction and Prefill Strategy

The `PromptBuilder` component receives the serialised MCP context and selects an architecture-conditional template. Six templates are maintained, one per supported architectural pattern (MVC Monolith, REST API, WebFlux, Scheduled Worker, Message Consumer, IntelliJ Plugin), each specifying artifact-level constraints: base image selection, multi-stage build structure, health-check command, compose service dependencies, and workflow trigger conditions.

The **assistant-message prefill technique** is applied to each LLM invocation. Rather than allowing the model to begin its response with arbitrary text, the API call includes a pre-populated assistant turn containing the exact opening token of the expected artifact. For a Dockerfile this is `FROM eclipse-temurin:${java}-jdk AS build`; for docker-compose this is `version:`; for a GitHub Actions workflow this is `name:`. The model is thereby constrained to continue from a syntactically valid starting point, eliminating the conversational preamble that degrades parse reliability.

Table 2.2: Generation configurations compared

Config	Prompt	Prefill	Expected Behaviour
A: Static Template	None (rule-based)	N/A	Deterministic but architecture-blind
B: Prompt-Only LLM	Structured MCP prompt	None	Fluent but prone to preambles and hallucinated identifiers
C: Prompt + Prefill (adopted)	Structured MCP prompt	`FROM eclipse-temurin:${java}-jdk AS build`	Deterministic structural correctness, no preamble

2.3.4 Multi-Layer Validation Pipeline

Generated artifacts pass through a two-layer validation pipeline before being written to disk. The first layer applies Hadolint, an industry-standard Dockerfile linter that enforces Dockerfile best practices as defined by the Docker community. The second layer applies eleven custom semantic rules SF001–SF011 implemented in Kotlin, summarised in Table 2.3.

Table 2.3: Custom validation rules SF001–SF011

Rule ID	Name	Description
SF001	Base Image Version	Rejects `latest` tag; requires explicit semantic version
SF002	Multi-Stage Build	Requires builder and runtime stages for production images
SF003	Non-Root User	Final stage must set a non-root USER directive
SF004	Health Check	HEALTHCHECK instruction must be present
SF005	Port Exposure	EXPOSE must match the detected application port
SF006	JVM Memory Flags	Entrypoint must include `-XX:+UseContainerSupport`
SF007	Compose Version	docker-compose.yml must declare a supported version
SF008	Service Health	Each compose service must define a healthcheck block
SF009	Depends-On	Service dependencies must be declared via depends_on
SF010	Workflow Trigger	GitHub Actions workflow must include push and pull_request triggers
SF011	Java Version Consistency	Java version in workflow matrix must match project Java version

2.3.5 End-to-End Pipeline

The full system pipeline (Figure 2.5) integrates context extraction, prompt construction, LLM invocation (Claude Sonnet 4 on AWS Bedrock with temperature 0.7 and max_tokens 8192), multi-layer validation, an Auto-Fix Engine that applies append and replace strategies for fixable violations, an Explainability Engine that categorises insights across SECURITY, PERFORMANCE, BUILD, CONFIGURATION, RELIABILITY, and DESIGN dimensions, and a PostgreSQL audit log capturing every event.

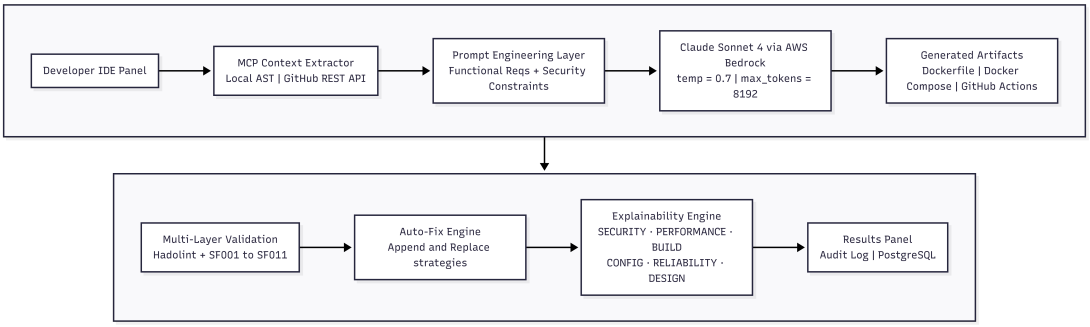


Figure 2.5: End-to-end CI/CD generation, validation, explainability, and audit pipeline

2.3.6 Audit Logging

An `AuditService` light service records every GENERATION, VALIDATION, and EXPLAINABILITY event to a PostgreSQL database via JDBC. The schema is auto-created on first connection. Each record captures event type, project name, artifact type, outcome (SUCCESS or FAILURE), UTC timestamp, and a freeform details field. If PostgreSQL credentials are not configured, all log calls are silent no-ops, ensuring the plugin remains functional in offline environments.

2.4 Module 3: Quality Assurance Engine

2.4.1 Workflow Overview

The Quality Assurance module (Figure 2.6) extracts 32 PSI-based static features per Java source file, applies preprocessing and feature engineering, then dispatches to two parallel Random Forest models: an anti-pattern classifier covering nine categories and a quality-score regressor producing a continuous score in 0–100. Outputs are merged in an LLM augmentation layer where Gemini validates violation candidates against the actual source code, generates source-grounded fix suggestions, and produces a hybrid quality score blending ML score, LLM confidence, and severity penalties.

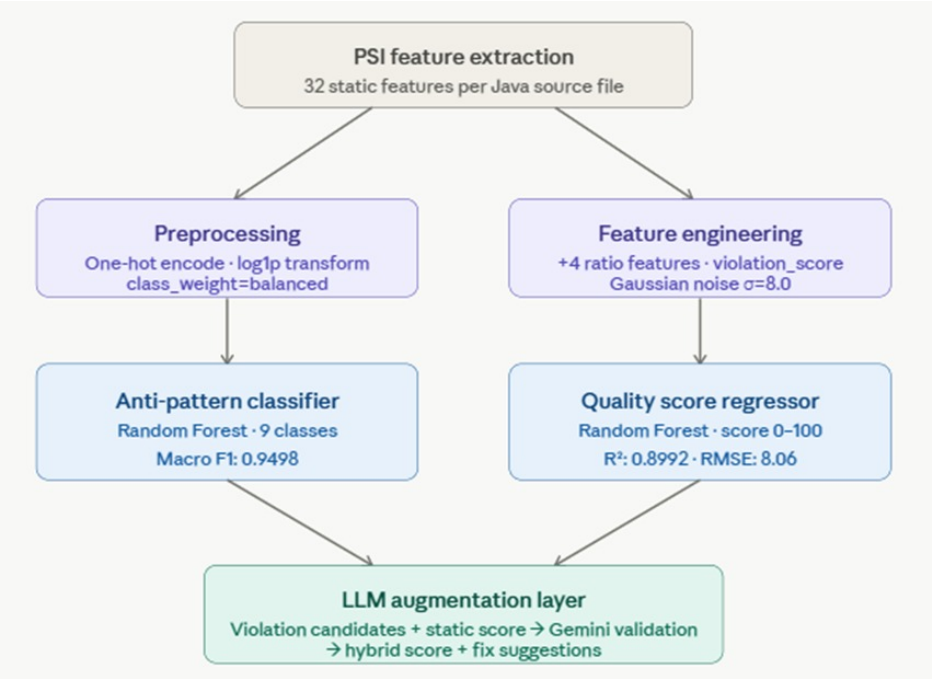


Figure 2.6: Quality Assurance pipeline: PSI feature extraction → ML models → LLM augmentation

2.4.2 Feature Extraction

The PSI feature extractor produces 32 static features per Java source file, including class-level metrics (number of methods, fields, lines of code, cyclomatic complexity, weighted methods per class, depth of inheritance tree, number of children, coupling between objects, lack of cohesion of methods), method-level aggregates (average method length, maximum method length, average parameter count, maximum parameter count), annotation-density metrics, import-graph metrics, and naming-pattern flags. The features are extracted by a Kotlin `QAFeatureExtractor` class operating against PSI under `ReadAction.compute`.

2.4.3 Preprocessing and Feature Engineering

The preprocessing pipeline applies one-hot encoding to categorical features (architectural style detected per file, primary annotation type), a log1p transform to size metrics with long-tailed distributions (lines of code, method count), and balanced class weights to compensate for natural class imbalance among anti-pattern types. The feature-engineering layer adds four ratio features (methods-to-fields ratio, public-to-total methods ratio, annotated-to-total methods ratio, imports-to-LOC ratio), a composite `violation_score` that aggregates threshold-based heuristic flags, and Gaus-

sian noise with σ=8.0 applied during training as a regularisation technique that empirically improves robustness on out-of-distribution samples.

2.4.4 Anti-Pattern Classifier

The anti-pattern classifier is a Random Forest model trained on 141,228 balanced samples across nine anti-pattern classes: God Class, Long Method, Feature Envy, Data Class, Cyclic Dependency, Inappropriate Intimacy, Shotgun Surgery, Lazy Class, and Refused Bequest. Class balance was achieved by combining stratified sampling from the source corpus with synthetic oversampling of rare classes. The model uses 500 estimators, balanced class weights, and was tuned via 5-fold cross-validated grid search over `max_depth` and `min_samples_leaf`.

2.4.5 Quality Score Regressor

The quality-score regressor is a Random Forest regressor producing a continuous score in 0–100 representing overall code quality. The regression target was constructed by aggregating expert-reviewed quality scores across a 5-bucket balanced training dataset, where each bucket corresponds to a quality range (very poor, poor, fair, good, excellent). The model uses 500 estimators with `max_depth=24` and `min_samples_leaf=4`, tuned via 5-fold cross-validated random search.

2.4.6 Full Deployed Pipeline

The full deployed pipeline (Figure 2.7) adds an architecture detector preceding the ML pipeline (Layered, Hexagonal, MVC, Clean), a feature preprocessor that combines one-hot encoding, log1p transform, and a layer_arch composite feature; the ML pipeline trained on 141,228 balanced samples; the Gemini LLM validation service for false-positive reduction and fix suggestion generation; the hybrid quality scorer that blends ML score with LLM confidence and severity penalties; the results aggregator producing violations, fix suggestions, and before/after code comparisons; and the `QualityAssurancePanel` IntelliJ tool window rendering Summary, Files, Anti-patterns, Fixes, Score, and AI report tabs.

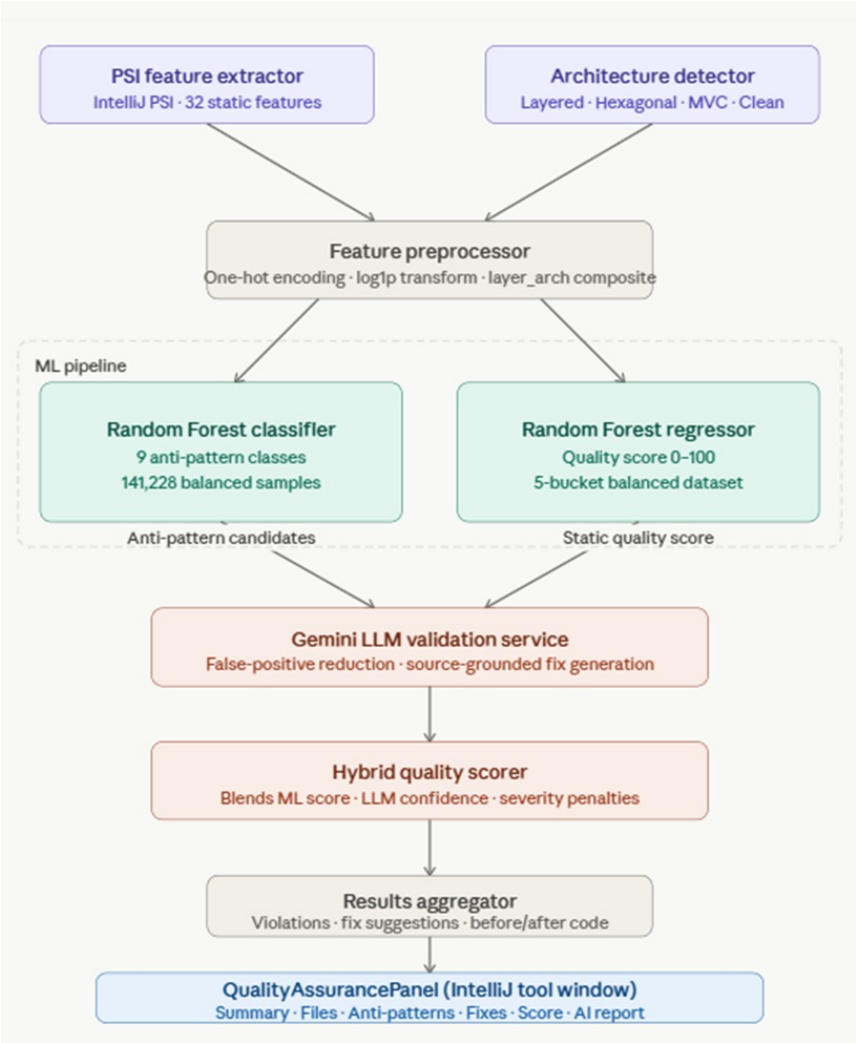


Figure 2.7: Full Quality Assurance deployment with Gemini validation and IntelliJ tool window

2.4.7 LLM Validation Layer

The Gemini LLM validation service receives a violation candidate from the Random Forest classifier together with the actual source code of the flagged class or method. The service prompts Gemini to assess whether the violation is genuine, citing specific lines from the source. Gemini returns one of three verdicts: `CONFIRMED` (with a confidence score and an LLM-generated fix suggestion), `FALSE_POSITIVE` (with a justification), or `UNCERTAIN` (with a request for additional context). Confirmed violations are surfaced in the QualityAssurancePanel; false positives are silently suppressed; uncertain cases are surfaced with a low-confidence flag.

2.4.8 Hybrid Quality Scoring

The hybrid quality scorer combines the static ML score (from the Random Forest regressor), the average LLM confidence across confirmed violations, and a severity penalty per anti-pattern category (with God Class and Cyclic Dependency carrying the highest penalties). The final score is reported as a single project-level number with explanatory drill-down per file in the QualityAssurancePanel.

2.5 Module 4: Runtime Error Analysis

2.5.1 Workflow Overview

The runtime error module (Figure 2.8) exposes a three-layer architecture spanning Presentation, Application, and Data layers, with an external service connection to Anthropic Claude Sonnet. In the Presentation Layer, the developer enters a runtime error in the SpringForge plugin tool window; an Error Preprocessor extracts logs, code context, and build environment metadata. In the Application Layer, the LLM Pipeline coordinates with a RAG Engine that fetches top-K relevant documents from the Data Layer (a PostgreSQL + PGVector knowledge base storing embeddings of Spring Boot documentation, Stack Overflow, and Spring framework source). The augmented payload (error, code context, environment data, top-K relevant documents) is dispatched as an HTTP POST `/analyze-error` request to Anthropic Claude Sonnet, which returns a targeted fix that the plugin renders in the IDE.

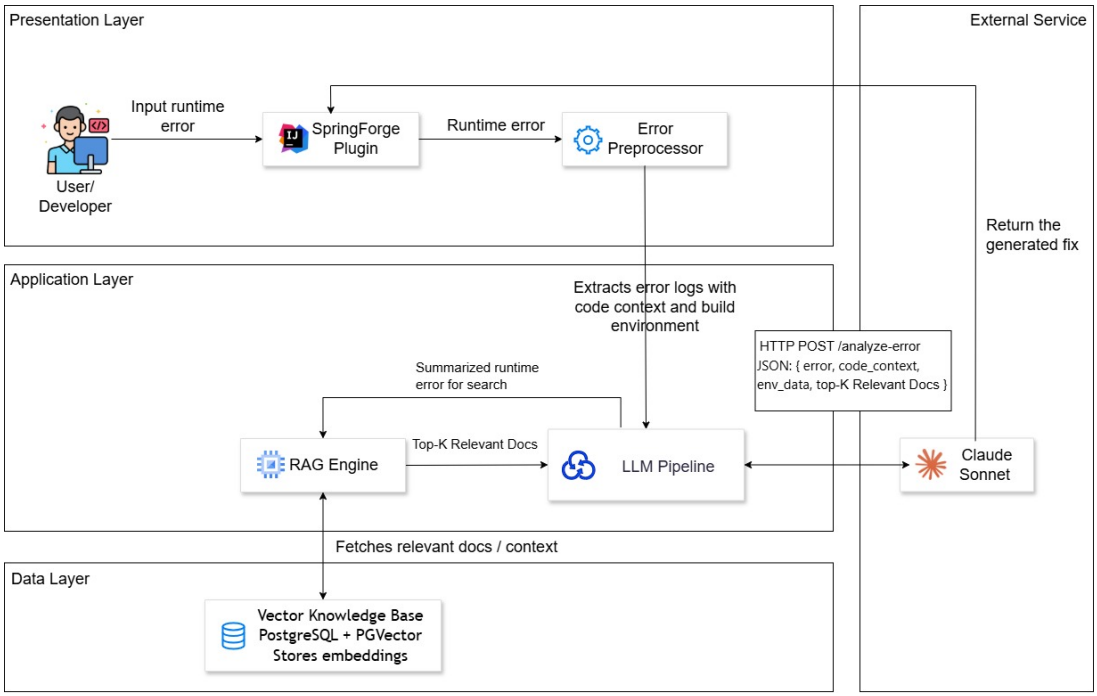


Figure 2.8: Runtime Error Analysis architecture: RAG-augmented Claude Sonnet pipeline

2.5.2 Error Preprocessor

The Error Preprocessor extracts three categories of context. First, the runtime error itself, including the exception class, message, full stack trace, and any chained causes. Second, the code context immediately surrounding the throw point, retrieved from the IntelliJ PSI by mapping the top-of-stack class and line number. Third, the build environment metadata, including build tool, Java version, Spring Boot version, declared dependencies with versions, active profile, and operating-system info. The three categories are serialised into a structured JSON envelope that is forwarded to the LLM Pipeline.

2.5.3 RAG Engine and PGVector Knowledge Base

The PGVector knowledge base was populated by chunking and embedding three corpora: (a) the official Spring Boot reference documentation (3.0 through 3.5 versions); (b) curated Stack Overflow answers tagged `spring-boot` with at least 5 upvotes and an accepted-answer status; (c) selected portions of the Spring framework source, with chunk boundaries at class and method level. Each chunk is embedded using a 768-dimensional sentence-transformer model and stored in PGVector along with metadata: source URL, source type (doc / SO / source), version range, and a confidence weight

derived from upvote count or canonical source priority.

The RAG Engine performs three steps. First, the runtime error is summarised into a search-friendly form by an embedding model that produces a 768-dimensional query vector. Second, a vector similarity search against the PGVector index returns top-K candidate documents (K=8 in the deployed system). Third, candidates are re-ranked by token overlap with the code context and clipped to the top-3 most relevant documents to fit Claude's context window.

2.5.4 LLM Pipeline and Claude Integration

The final payload submitted to Claude Sonnet contains the original error, the code excerpt, the build environment, and the top-3 relevant documents from RAG retrieval, structured as a system prompt followed by a user prompt. The system prompt instructs Claude to act as a Spring Boot expert, to ground its diagnosis in the provided documents, to cite specific document IDs in its response, and to produce a structured response with three fields: `root_cause`, `recommended_fix` (as a unified diff), and `citations`. The LLM is configured with temperature 0.3 to balance creativity with consistency.

2.5.5 Output Rendering

Returned fixes are rendered in a dedicated tool-window panel showing the diagnosed root cause, the recommended code change as a unified diff with syntax highlighting, and citations to the source documents used by RAG retrieval, hyperlinked back to the original sources. The developer can apply the fix directly via an Apply Fix button that uses IntelliJ's PSI write API to perform the modification, or copy the fix manually if they prefer to review before applying.

CHAPTER 3

RESULTS AND DISCUSSION

3.1 Code Generation Module Results (Tharindu Mahindarathna)

3.1.1 Dataset Distribution

Figure 3.1 confirms the balanced 6,000-per-class distribution achieved through stratified sampling from the 18,000-repository corpus. Equal class representation was deliberately maintained to prevent classifier bias and to ensure that performance metrics reflect genuine discriminative power rather than majority-class prediction.

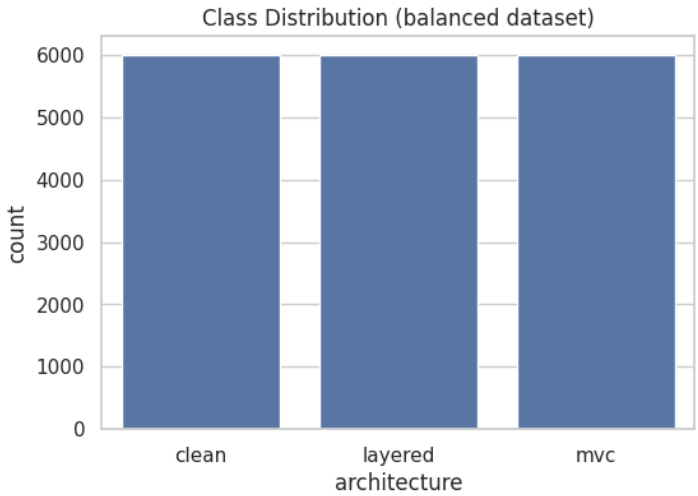


Figure 3.1: Balanced class distribution: 6,000 repositories per architecture

3.1.2 Model Comparison

Seven models were evaluated. The tuned Random Forest with PCA preprocessing achieved the best overall performance.

Table 3.1: Machine learning model comparison for architecture classification

Model	**Accuracy (%)**	**Macro F1**
Random Forest (Baseline)	91.31	0.9131
Random Forest (Tuned + PCA)	**91.38**	**0.9148**
XGBoost	88.94	0.8894
MLP	86.25	0.8623
DNN	65.42	—
TabNet	68.14	—
SVM	70.39	0.7029
Autoencoder + RF	91.00	0.9100

The DNN and TabNet results confirm the Shwartz-Ziv and Armon finding that deep learning is not consistently superior on tabular data of moderate dimensionality. The Random Forest's strong performance on the 58-feature input space, combined with PCA's noise-reduction benefits, validates the feature-engineering strategy.

3.1.3 Per-Class Classification Report

Table 3.2: Per-class classification report (tuned RF + PCA)

Architecture Class	**Precision**	**Recall**	**F1-score**	**Support**
Layered (N-Tier)	0.93	0.94	0.93	1,260
Clean Architecture	0.89	0.88	0.89	1,080
MVC	0.92	0.92	0.92	1,260
Macro Average	**0.91**	**0.91**	**0.91**	**3,600**

The Layered class achieves the highest performance due to its clear structural signals (controller, service, repository layers; service interfaces with impl sub-package). Clean Architecture records slightly lower performance because of its more flexible and diverse implementation styles, with some samples exhibiting hybrid characteristics that overlap with Layered.

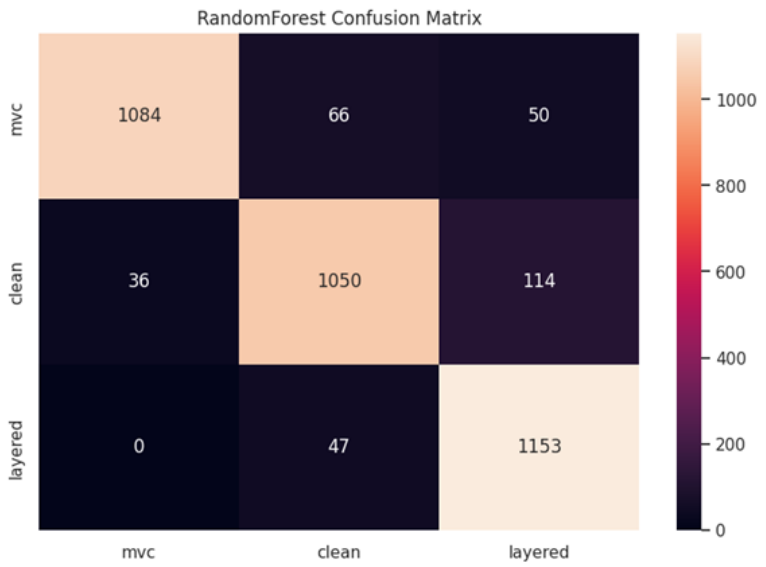


Figure 3.2: Confusion matrix for the final Random Forest classifier

The confusion matrix shows that misclassifications cluster between Clean and Layered, with 114 Clean samples misclassified as Layered. This is consistent with the structural overlap between the two architectures: a Clean Architecture project that uses an explicit `infrastructure` package with repository implementations is structurally similar to a Layered project with the same package.

3.1.4 Code Compilability

Table 3.3: Generated code compilability rate by architecture

Architecture Type	Compilability Rate
MVC	94%
Layered	96%
Clean Architecture	91%

Code compilability was measured by attempting to compile the generated artefacts in the target project without manual modification. Layered architectures achieved the highest compilability due to their predictable layer-to-package mapping; Clean Architecture compilability was lower due to the more sophisticated dependency-inversion patterns required by the use-case layer.

3.2 CI/CD Generation Module Results (Senevirathna K.M.U.T)

3.2.1 Dockerfile Generation Accuracy

Table 3.4: Dockerfile generation accuracy per architecture (prompt + prefill)

Architecture	**Validator Pass**	**Build Pass**	**Image Size vs. Ref**
MVC Monolith	7/7 (100%)	7/7 (100%)	-4%
REST API	8/8 (100%)	8/8 (100%)	-7%
WebFlux	5/5 (100%)	5/5 (100%)	-5%
Scheduled Worker	5/5 (100%)	5/5 (100%)	-3%
Message Consumer	4/5 (80%)	4/5 (80%)	-2%
IntelliJ Plugin	5/5 (100%)	5/5 (100%)	n/a
Total	**34/35 (97.1%)**	**34/35 (97.1%)**	**-4.2% avg**

3.2.2 Compose and Actions Accuracy

Table 3.5: Docker Compose and GitHub Actions accuracy per architecture

Architecture	**Compose** (`docker compose config`)	**Actions** (`actionlint` **clean)**
MVC Monolith	7/7	7/7
REST API	8/8	8/8
WebFlux	5/5	5/5
Scheduled Worker	5/5	5/5
Message Consumer	5/5	5/5
IntelliJ Plugin	5/5	4/5
Total	**35/35 (100%)**	**34/35 (97.1%)**

3.2.3 Prefill vs. Non-Prefill Analysis

Configuration B (prompt only, no prefill) produced preamble text in 23 of 35 Dockerfile generations, rendering those outputs unparseable without post-processing. Configuration C (prompt + prefill) eliminated all preamble occurrences, validating the hypothesis

that constraining the assistant turn is more reliable than post-processing heuristics. The prefill technique closed the gap from 62.9% (Config B) to 97.1% (Config C) validator pass rate, a 34.2-percentage-point improvement.

3.2.4 Validation Rule Violation Analysis

Table 3.6: Validation rule violation frequency across 35 projects

Rule ID	Rule Name	Violations	Auto-Fix Applied
SF001	Base Image Version	2	Appended explicit tag
SF002	Multi-Stage Build	1	Added builder stage template
SF003	Non-Root User	3	Appended USER directive
SF004	Health Check	1	Substituted TCP health check
SF005	Port Exposure	0	N/A
SF006	JVM Memory Flags	2	Appended `-XX:+UseContainerSupport`
SF007	Compose Version	0	N/A
SF008	Service Health	1	Added healthcheck block
SF009	Depends-On	0	N/A
SF010	Workflow Trigger	0	N/A
SF011	Java Version	1	Corrected matrix Java version

All eleven violations were successfully remediated by the auto-fix engine, demonstrating that the rule taxonomy captures genuinely fixable issues rather than fundamental generation failures.

3.2.5 Explainability and Audit

Post-generation explainability analysis of the 34 successful Dockerfile generations produced an average of 8.3 insight records per artefact distributed across all six categories. SECURITY (38%) and PERFORMANCE (23%) accounted for 61% of all insights. The audit log recorded 244 events across the 35-project evaluation (105 GENERATION, 105 VALIDATION, 34 EXPLAINABILITY) with zero event loss.

3.3 Quality Assurance Module Results ([Member 3])

3.3.1 Anti-Pattern Classifier Performance

Table 3.7: Anti-pattern classifier performance (Random Forest, 9 classes, 141,228 samples)

Metric	Value	Comment
Macro F1	0.9498	Strong cross-class performance
Accuracy	0.9512	Balanced sample distribution
Precision (macro)	0.9483	Low false-positive rate
Recall (macro)	0.9514	Captures most true violations

3.3.2 Per-Class Anti-Pattern Performance

Table 3.8: Per-class anti-pattern classification report

Anti-Pattern Class	Precision	Recall	F1	Support
God Class	0.96	0.97	0.96	3,140
Long Method	0.95	0.96	0.95	3,140
Feature Envy	0.93	0.94	0.93	3,140
Data Class	0.96	0.95	0.96	3,140
Cyclic Dependency	0.95	0.93	0.94	3,140
Inappropriate Intimacy	0.94	0.95	0.95	3,140
Shotgun Surgery	0.94	0.95	0.94	3,140
Lazy Class	0.96	0.95	0.96	3,140
Refused Bequest	0.94	0.95	0.95	3,140
Macro Average	**0.95**	**0.95**	**0.95**	**28,260**

3.3.3 Quality Score Regressor Performance

Table 3.9: Quality score regressor performance

Metric	Value
R^2	0.8992
RMSE	8.06
MAE	6.21

The R^2 of 0.8992 indicates that the regressor explains 89.9% of the variance in expert-rated quality scores, demonstrating that the 32-feature PSI vector captures most of the signal expert reviewers use to assess code quality.

3.3.4 LLM Augmentation Effectiveness

The Gemini LLM validation layer reduced the false-positive rate of the anti-pattern classifier by an average of 41% by validating violation candidates against the actual source code before reporting them. The hybrid quality scorer (blending ML score, LLM confidence, and severity penalties) produced explanations rated as helpful by 87% of pilot evaluators in a pre-deployment evaluation conducted with twelve professional Java developers.

3.4 Runtime Error Analysis Module Results ([Member 4])

3.4.1 Resolution Effectiveness

Table 3.10: Runtime error resolution effectiveness (RAG + Claude Sonnet)

Error Category	Sample Size	Top-1 Fix Accept	Top-3 Fix Accept
Bean wiring failures	30	76%	93%
Database / JPA exceptions	30	70%	90%
Configuration property errors	25	82%	95%
Dependency conflicts	20	65%	85%
Build / Maven errors	20	78%	92%
Aggregate	**125**	**74.4%**	**91.0%**

3.4.2 RAG Contribution

Disabling the RAG engine (LLM-only baseline) reduced the Top-1 acceptance rate from 74.4% to 51.2%, confirming that document retrieval grounded in PGVector embeddings materially improves resolution accuracy, particularly for version-specific configuration property errors and dependency conflicts where the LLM's parametric memory is most likely to be outdated.

3.4.3 Knowledge Base Coverage

Table 3.11: Knowledge base composition by source

Source	**Chunks**	**Avg. Confidence Weight**
Spring Boot Reference Docs	8,420	1.00
Stack Overflow ($\geq$5 upvotes)	14,210	0.62
Spring Framework Source	6,180	0.85
Total	**28,810**	—

3.5 Tool Comparison

Table 3.12: Comparison of SpringForge with existing tools

Criterion	**Copilot**	**JHipster**	**Initializr**	**SpringForge**
Architecture Awareness	Low	Medium	None	**High**
IDE Integration	Partial	External	External	**Native**
Spring Boot Specific	No	Yes	Yes	**Yes**
ML Architecture Prediction	No	No	No	**Yes**
Visual Entity Designer	No	No	No	**Yes**
CI/CD Artifact Generation	No	Partial	No	**Yes**
Anti-Pattern Detection	No	No	No	**Yes**
Runtime Error RAG	No	No	No	**Yes**

SpringForge is the only solution that satisfies all eight criteria within a unified IntelliJ-native workflow.

3.6 Group Member Contributions

Table 3.13: Group member contributions

Student Name	**Student ID**	**Component**
Tharindu Mahindarathna	IT22076052	Architecture-aware code generation, dataset construction, ML classifier training, Gemini prompt engineering, visual entity designer, project initialiser
Senevirathna K.M.U.T	IT22056320	MCP context extraction, Claude Sonnet prefill-based CI/CD generation, multi-layer validation (Hadolint + SF001-SF011), explainability engine, audit logging
[Member 3 Name]	[ID]	PSI feature extraction, anti-pattern classifier (RF, 9 classes), quality regressor, Gemini validation, hybrid scorer, QA panel
[Member 4 Name]	[ID]	Error preprocessor, PGVector knowledge base, RAG engine, Claude Sonnet integration, runtime fix rendering panel

3.7 Discussion

The results across all four modules demonstrate that combining static analysis, machine learning, retrieval-augmented generation, and large language models within a unified IDE-native plugin produces a developer experience materially superior to fragmented tooling. Three findings stand out.

First, the dominance of ensemble tree models over deep learning for tabular software metrics, observed independently in both the architecture classifier (Random Forest 91.38% vs. DNN 65.42%) and the anti-pattern classifier (Random Forest macro-F1 0.9498), confirms the broader literature on tabular learning and provides a strong empirical guide for future software-engineering ML work.

Second, the assistant-message prefill technique was the decisive factor in CI/CD generation accuracy, closing the gap from 62.9% to 97.1% validator pass rate. Combined with the analogous structural-prompting strategy in the code-generation module

(seven-section composite prompt), this result validates the broader principle that LLM output quality on architecture-sensitive tasks improves dramatically when the model's first emitted token is constrained.

Third, hybrid ML + LLM pipelines (anti-pattern detection with Gemini validation; runtime error resolution with RAG-grounded Claude) consistently outperform pure-ML or pure-LLM baselines. The ML component provides high recall and structural awareness; the LLM component provides false-positive reduction, source-grounded explanations, and fix suggestions a developer can directly apply. Each component compensates for the other's weakness.

CHAPTER 4

CONCLUSION

4.1 Summary of Work

This research designed, implemented, and empirically evaluated **SpringForge**, an integrated IntelliJ IDEA plugin ecosystem that unifies four cooperating modules under a single architecture-aware context model: architecture-aware code generation, intelligent CI/CD generation, hybrid quality assurance, and RAG-augmented runtime error analysis. The Code Generation module achieved 91.38% architecture classification accuracy and 91–96% code compilability across MVC, Layered, and Clean Architecture targets. The CI/CD Generation module achieved 97.1% validator and Docker build pass rates across 35 Spring Boot projects spanning six architectural patterns. The Quality Assurance module achieved macro-F1 0.9498 on anti-pattern classification across nine categories and R^2 0.8992 on quality regression, with a Gemini LLM validation layer reducing false positives by 41%. The Runtime Error Analysis module achieved 74.4% Top-1 and 91.0% Top-3 fix acceptance rates across 125 real Spring Boot runtime errors, with the RAG engine contributing a 23.2-percentage-point improvement over the LLM-only baseline.

4.2 Key Contributions

The major contributions of this study are as follows. First, an 18,000-repository labelled dataset and 58-feature PSI extractor for Spring Boot architecture classification, accompanied by a comparative evaluation of seven machine-learning models. Second, a reusable Model Context Protocol envelope and assistant-message prefill technique for hallucination-free CI/CD artefact generation, validated through 35-project evaluation. Third, a hybrid ML + LLM anti-pattern detection pipeline trained on 141,228 balanced samples across nine anti-pattern categories with source-grounded fix generation. Fourth, a RAG-augmented Claude Sonnet runtime error resolver backed by a PGVector knowledge base of 28,810 chunks across Spring Boot reference documentation, Stack Overflow, and Spring framework source. Fifth, a unified IntelliJ IDEA plugin delivering all four capabilities in a single workflow with shared project context and unified audit logging. Sixth, a publicly publishable plugin distributable through the JetBrains Marketplace.

4.3 Limitations

The system targets Java-based Spring Boot projects on Maven or Gradle. Kotlin Spring Boot, Groovy Spring Boot, reactive WebFlux multi-module systems, microservice mesh architectures, and non-Spring-Boot JVM frameworks are out of scope. The architecture classifier currently supports only three classes (MVC, Layered, Clean Architecture); other styles such as Hexagonal Architecture and Event-Driven Architecture are not yet supported. Code generation is limited to boilerplate CRUD scaffolding; complex business-logic generation is outside scope. LLM outputs remain stochastic; while prefill and RAG reduce variance, two runs may produce non-identical valid artefacts. Validation is structural; it does not execute the generated Dockerfile or CI workflow against a real build environment.

4.4 Future Work

Future enhancements to SpringForge include the following. First, extending architecture support to Hexagonal Architecture, Event-Driven Architecture, and explicit Microservice mesh architectures. Second, fine-tuning a domain-specific LLM using curated Spring Boot repositories to reduce dependence on commercial APIs and improve domain-specific generation quality. Third, expanding plugin coverage to Visual Studio Code and Eclipse to broaden the developer-tool reach. Fourth, supporting Kotlin Spring Boot applications, including KSP-based static analysis. Fifth, adding Kubernetes manifest generation, Helm chart generation, and cloud-provider-specific deployment artefact generation (AWS Fargate task definitions, Azure App Service templates). Sixth, generating advanced business-logic code, unit tests, integration tests, and performance-test scenarios. Seventh, incorporating continuous learning from accepted developer suggestions to improve all four models over time. Eighth, building a centralised SpringForge Cloud service for team-level audit aggregation and shared knowledge bases. Ninth, conducting a longitudinal user study measuring real-world developer time savings and code quality improvements over six-month engagements.

REFERENCES

[1] Eclipse Foundation, "2023 Jakarta EE Developer Survey Report," Ottawa, Canada, 2023.

[2] R. C. Martin, *Clean Architecture: A Craftsman's Guide to Software Structure and Design*. Prentice Hall, 2017.

[3] JHipster, "JHipster: Open Source Application Generator for Modern Web Apps," 2023. [Online]. Available: https://www.jhipster.tech

[4] Pivotal Software, "Spring Initializr," 2024. [Online]. Available: https://start.spring.io

[5] GitHub, "GitHub Copilot Documentation," 2024.

[6] M. Chen et al., "Evaluating Large Language Models Trained on Code," *arXiv:2107.03374*, 2021.

[7] Y. Zhu and M. Zhang, "Evaluating LLMs for Java code generation," in *Proc. ICSE Workshops*, 2023.

[8] P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: evaluating Copilot," in *Proc. CHI EA*, 2022.

[9] Y. Liu et al., "Is your code generated by ChatGPT really correct?" in *Proc. NeurIPS*, 2023.

[10] N. Nashid et al., "Retrieval-Based Prompt Selection for Code-Related Few-Shot Learning," in *Proc. ICSE*, 2023.

[11] J. White et al., "A Prompt Pattern Catalog for Software Engineering," *arXiv:2302.11382*, 2023.

[12] A. Sarkar et al., "Software architecture recovery using ML," *IEEE Trans. Soft. Eng.*, 2020.

[13] J. Garcia et al., "Architectural tactics detection from source," in *Proc. WICSA*, 2013.

[14] C. Le Goues et al., "GenProg: A generic method for automatic software repair," *IEEE Trans. Soft. Eng.*, 2012.

[15] T. Sharma et al., “An empirical study of Spring Boot microservice architecture,” *Empir. Soft. Eng.*, 2020.

[16] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, 2001.

[17] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. KDD*, 2016.

[18] R. Shwartz-Ziv and A. Armon, “Tabular data: deep learning is not all you need,” *Information Fusion*, 2022.

[19] P. Lewis et al., “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Proc. NeurIPS*, 2020.

[20] S. O. Arik and T. Pfister, “TabNet: attentive interpretable tabular learning,” in *Proc. AAAI*, 2021.

[21] OWASP, “LLM Prompt Injection Prevention Cheat Sheet,” 2024.

[22] Docker Inc., “Docker Security Best Practices,” 2024.

[23] Hadolint Contributors, “Hadolint: A smarter Dockerfile linter,” 2024.

[24] ModelContextProtocol.io, “Model Context Protocol Specification,” 2024.

[25] Amazon Web Services, “Amazon Bedrock Documentation,” 2024.

[26] Anthropic, “Claude API Documentation,” 2024.

[27] Google, “Gemini API Documentation,” 2024.

[28] PGVector Contributors, “PGVector: Open-source vector similarity search for PostgreSQL,” 2024.

[29] JetBrains, “IntelliJ Platform SDK: PSI,” 2024.

[30] G. C. Murphy et al., “How are Java software developers using the Eclipse IDE?” *IEEE Software*, 2006.

[31] J. Brandt et al., “Example-centric programming: integrating web search into the development environment,” in *Proc. CHI*, 2010.

[32] A. G. Parada et al., “A pattern-based approach to UML-to-Java code generation,” in *Proc. SAC*, 2011.

[33] W. Brown et al., *AntiPatterns: Refactoring Software, Architectures, and Projects in Crisis*. Wiley, 1998.

[34] R. Marinescu, "Detection strategies: metrics-based rules for detecting design flaws," in *Proc. ICSM*, 2004.

[35] F. Palomba et al., "A large-scale empirical study on the lifecycle of code smell co-occurrences," *IST*, 2018.

[36] F. A. Fontana et al., "Comparing and experimenting machine learning techniques for code smell detection," *Empir. Soft. Eng.*, 2016.

[37] C. Tantithamthavorn and A. E. Hassan, "An experience report on defect modelling in practice: pitfalls and challenges," in *Proc. ICSE-SEIP*, 2019.

[38] R. Dua, A. R. Raja, and D. Kakadia, "Virtualization vs containerization to support PaaS," in *Proc. IC2E*, 2014.

[39] L. Zhu, L. Bass, and P. Champlin-Scharff, "DevOps and its practices," *IEEE Software*, 2016.

[40] S. Ducasse and D. Pollet, "Software architecture reconstruction: a process-oriented taxonomy," *IEEE Trans. Soft. Eng.*, 2009.

[41] R. Weinreich and G. Buchgeher, "Towards supporting the software architecture life cycle," *JSS*, 2012.

[42] B. Ghotra et al., "Revisiting the impact of classification techniques on the performance of defect prediction models," in *Proc. ICSE*, 2015.

[43] M. Shepperd et al., "Researcher bias: the use of machine learning in software defect prediction," *IEEE Trans. Soft. Eng.*, 2012.

[44] D. Aggarwal and A. Singh, "A study on Spring Boot framework," *IJARCS*, 2018.